ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 9

(N4. Լագրանժի հավասարում)

Թեմա՝ Դասի դինամիկ անդամներով խնդիրներ՝ մաթեմատիկական անալիզի թեմաներով

Աշխատանքի նպատակ

Ուսումնասիրել դինամիկ հիշողության կառավարումը C++ դասերում, իրականացնել «Երեքի կանոնը» (Rule of Three՝ կոնստրուկտոր, պատճենող կոնստրուկտոր, վերագրման օպերատոր և դեստրուկտոր) և կատարել Լագրանժի հավասարման պարամետրական լուծումների հաշվարկ:

Խնդրի պահանջ Տրված է Լագրանժի հավասարումը՝ `y = 2xp + p^2`, որտեղ `p = y'`:

Պարամետրական լուծումներն են՝

- `x = C / p^2 - (2/3)p`
- `y = 2C / p - p^2 / 3`

Պահանջվում է.

- Ընդունել `C = 2`, `p = 1 + 0.05i` (`i = 0...20`):
- Ստեղծել `LagrangeSolver` դասը, որի `private` դաշտերում նախատեսված են `double*` դինամիկ զանգվածներ (`p`, `x`, `y`-ի համար) և դրանց չափը:
- Իրականացնել պարամետրերով կոնստրուկտոր, պատճենող կոնստրուկտոր (խորքային պատճենում), վերագրման օպերատոր և դեստրուկտոր:
- Հաշվել կետերը և գտնել ամենամեծ `y`-ն:
- Արդյունքները արտածել աղյուսակի տեսքով:

Տեսական մաս

- **Դինամիկ հիշողություն դասերում.** Երբ դասի դաշտերը հանդիսանում են ցուցիչներ (`double*`), հիշողության հատկացումը կատարվում է `new[]` օպերատորով, իսկ ազատումը՝ `delete[]`-ով դեստրուկտորում:
- **Երեքի կանոնը (Rule of Three).** Եթե դասն ունի դինամիկ հիշողություն, ապա պարտադիր է իրականացնել.
    - **Դեստրուկտոր (`~LagrangeSolver()`)**՝ հիշողության ազատման համար:
    - **Պատճենող կոնստրուկտոր (`LagrangeSolver(const LagrangeSolver&)`)**՝ խորքային պատճենման (Deep Copy) համար, որպեսզի ցուցիչները չհղվեն նույն հիշողության վրա:
    - **Վերագրման օպերատոր (`operator=`)**՝ առկա հիշողությունը մաքրելու և նոր տվյալները խորքային պատճենելու համար:

C++ Ծրագրի կոդը

```
#include <iostream>

#include <iomanip>

#include <cmath>

using namespace std;

class LagrangeSolver {

private:

    double* pValues;

    double* xValues;

    double* yValues;

    int count;

    double C;

    void allocateMemory(int n) {
```

```
        count = n;
        pValues = new double[count];
        xValues = new double[count];
        yValues = new double[count];
    }
    void freeMemory() {
        delete[] pValues;
        delete[] xValues;
        delete[] yValues;
        pValues = nullptr;
        xValues = nullptr;
        yValues = nullptr;
    }
public:
    // 1. Պարամետրերով կոնստրուկտոր
    LagrangeSolver(double cVal = 2.0, int steps = 21) {
        if (steps <= 0) {
            cout << "[Սխալ] Քայլերի քանակը անվավեր է: Սահմանվում է 21:\n";
            steps = 21;
        }
        C = cVal;
```

```
    allocateMemory(steps);
    for (int i = 0; i < count; ++i) {
        pValues[i] = 1.0 + 0.05 * i;
    }
}
// 2. Պատճենող կոնստրուկտոր (Deep Copy)
LagrangeSolver(const LagrangeSolver& other) {
    C = other.C;
    allocateMemory(other.count);
    for (int i = 0; i < count; ++i) {
        pValues[i] = other.pValues[i];
        xValues[i] = other.xValues[i];
        yValues[i] = other.yValues[i];
    }
}
// 3. Վերագրման օպերատոր (Deep Copy)
LagrangeSolver& operator=(const LagrangeSolver& other) {
    if (this != &other) {
        freeMemory();
        C = other.C;
        allocateMemory(other.count);
```

```
        for (int i = 0; i < count; ++i) {
            pValues[i] = other.pValues[i];
            xValues[i] = other.xValues[i];
            yValues[i] = other.yValues[i];
        }
    }
    return *this;
}
// 4. Դեստրուկտոր
~LagrangeSolver() {
    freeMemory();
}
// Հաշվարկային մեթոդ
void solve() {
    for (int i = 0; i < count; ++i) {
        double p = pValues[i];
        if (abs(p) < 1e-9) {
            xValues[i] = 0;
            yValues[i] = 0;
            continue;
        }
```

```
            xValues[i] = (C / (p * p)) - (2.0 / 3.0) * p;
            yValues[i] = (2.0 * C / p) - (p * p / 3.0);
        }
    }
    // Ամենամեծ y-ն գտնելու մեթոդ
    double getMaxY() const {
        if (count <= 0) return 0.0;
        double maxY = yValues[0];
        for (int i = 1; i < count; ++i) {
            if (yValues[i] > maxY) {
                maxY = yValues[i];
            }
        }
        return maxY;
    }
    // Աղյուսակի տեսքով արտածում
    void printTable() const {
        cout << fixed << setprecision(4);
        cout << "-----------------------------------------\n";
        cout << "|  i  |    p_i   |     x(p)    |    y(p)    |\n";
|---|---|---|---|"-----------------------------------------\n";
```

```
        for (int i = 0; i < count; ++i) {
            cout << "| " << setw(3) << i
                 << " | " << setw(8) << pValues[i]
                 << " | " << setw(11) << xValues[i]
                 << " | " << setw(10) << yValues[i] << " |\n";
        }
        cout << "-----------------------------------------\n";
    }
};
int main() {
    setlocale(LC_ALL, "Armenian");
    cout << "=== ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ № 9 (Վարիանտ N4) ===\n\n";
    LagrangeSolver solver1(2.0, 21);
    solver1.solve();
    cout << "Արդյունքների աղյուսակ:\n";
    solver1.printTable();
    cout << "\nԱմենամեծ y արժեքը (Max Y)։ " << solver1.getMaxY() << "\n\n";
    cout << "=== Խորքային պատճենման ստուգում (Rule of Three) ===\n";
    LagrangeSolver solver2 = solver1;
    cout << "Պատճենված օբյեկտի Max Y։ " << solver2.getMaxY() << "\n";
    return 0;
```

}

Հաշվարկված արդյունքների աղյուսակ

i	p_i	x(p)	y(p)
0	1.0000	1.3333	3.6667
1	1.0500	1.1141	3.4418
2	1.1000	0.9190	3.2330
3	1.1500	0.7449	3.0378
4	1.2000	0.5891	2.8538
5	1.2500	0.4491	2.6792
6	1.3000	0.3230	2.5126
7	1.3500	0.2089	2.3528
8	1.4000	0.1054	2.1989
9	1.4500	0.0113	2.0498
10	1.5000	-0.0741	1.9048
11	1.5500	-0.1517	1.7631
12	1.6000	-0.2222	1.6240
13	1.6500	-0.2864	1.4868
14	1.7000	-0.3447	1.3510
15	1.7500	-0.3978	1.2160
16	1.8000	-0.4461	1.0815
17	1.8500	-0.4899	0.9469
18	1.9000	-0.5298	0.8118
19	1.9500	-0.5661	0.6757
20	2.0000	-0.5990	0.5381

Եզրակացություն

- Դինամիկ հիշողության հետ աշխատելիս «Երեքի կանոնի» կիրառումը երաշխավորում է ծրագրի կայունությունը և կանխում հիշողության կրկնակի ազատման (`double free`) կամ հիշողության արտահոսքի (`memory leak`) խնդիրները:
- Ծրագրի աշխատանքի արդյունքում առավելագույն արժեքն է՝ **Max Y = 3.6667** (ստացվում է `i = 0`, `p = 1.0000` կետում):